Distributed Computing – Module Wise Descriptive Answers

MODULE 1 – INTRODUCTION TO DISTRIBUTED SYSTEMS

Q1. What is Distributed Computing? Explain the various issues in a distributed system.

Definition of Distributed Computing

Distributed computing is a computing model in which multiple autonomous computers communicate and coordinate with each other through a network to achieve a common objective. These systems appear to the user as a single integrated system even though the components are physically distributed.

A distributed system allows resource sharing, faster processing, improved reliability, scalability, and fault tolerance.

Examples

- Cloud Computing Systems
- Banking Systems
- Distributed Databases
- Google Search Engine
- Online Multiplayer Games
- Hadoop Clusters

Characteristics of Distributed Systems

1. Multiple autonomous computers.
2. Communication through message passing.
3. Resource sharing among systems.
4. Concurrency of components.
5. No global clock.
6. Independent failures may occur.

Issues in Distributed Systems

1. Heterogeneity

Different computers may use:

- Different operating systems
- Different hardware architectures
- Different programming languages
- Different communication protocols

Problem

Interoperability becomes difficult.

Solution

Middleware is used to provide uniform communication.

---

2. Transparency

The distributed nature of the system should be hidden from users.

Types of Transparency

- Access Transparency
- Location Transparency
- Migration Transparency
- Replication Transparency
- Concurrency Transparency
- Failure Transparency

Example

A user accessing Google Drive does not know where files are physically stored.

---

3. Scalability

The system should perform efficiently when:

- Number of users increases
- Number of resources increases
- Geographic distance increases

Challenges

- Network congestion
- Database bottlenecks
- Communication overhead

Solutions

- Replication
- Caching
- Decentralization

---

4. Security

Distributed systems are vulnerable because data travels through networks.

Security Threats

- Unauthorized access
- Data modification
- Data theft
- Impersonation attacks

Security Mechanisms

- Encryption
- Authentication
- Authorization
- Firewalls
- Digital Signatures

---

5. Fault Tolerance

Failure of one machine should not crash the entire system.

Types of Failures

- Process failure
- Communication failure
- Network failure
- Crash failure

Techniques Used

- Replication

- Checkpointing
- Backup systems
- Recovery protocols

---

6. Concurrency

Many users may access shared resources simultaneously.

Problems

- Data inconsistency
- Race conditions
- Deadlocks

Solutions

- Mutual exclusion
- Synchronization algorithms
- Locks and semaphores

---

7. Resource Management

Efficient management of:

- CPU
- Memory
- Storage
- Network bandwidth

Goal

Optimal utilization of system resources.

---

8. Communication Delays

Network communication is slower than local communication.

Problems

- Latency
- Packet loss
- Message ordering issues

Solutions

- Efficient protocols
- Data compression
- Caching

---

9. Naming and Directory Services

Resources distributed across systems require unique identification.

Example

DNS (Domain Name System)

---

10. Synchronization

Distributed systems lack a global clock.

Problems

- Event ordering
- Clock drift

Solutions

- Logical clocks
- Physical clock synchronization

Advantages of Distributed Computing

- Resource sharing
- High reliability
- Scalability
- Better performance
- Fault tolerance
- Geographic distribution

Disadvantages

- Complex design
- Security risks
- Network dependency

- Difficult debugging
- High communication overhead

Conclusion

Distributed computing enables multiple interconnected systems to work together efficiently. Although it provides scalability, reliability, and performance improvements, several challenges such as synchronization, security, scalability, and fault tolerance must be carefully managed.

\==========

Q2. What are the goals of a distributed system? Explain various system models of distributed computing.

Goals of Distributed Systems

1. Resource Sharing

Users should be able to access hardware, software, and data resources remotely.

Examples

- Shared printers
- Shared databases
- Cloud storage

---

2. Transparency

The distributed nature should remain hidden from users.

Types

- Access transparency
- Location transparency
- Replication transparency
- Failure transparency

---

3. Openness

The system should support:

- Standard protocols
- Interoperability
- Portability
- Extensibility

---

4. Scalability

The system should handle increasing:

- Users
- Resources
- Geographic distribution

Methods

- Replication
- Caching
- Distributed databases

---

5. Reliability and Fault Tolerance

System should continue functioning despite failures.

Methods

- Backup systems
- Replication
- Recovery techniques

---

6. Performance

Distributed systems improve performance using:

- Parallel processing
- Load balancing
- Distributed execution

---

7. Flexibility and Modularity

Components can be upgraded or modified independently.

System Models of Distributed Computing

1. Physical Model

Describes hardware organization.

Types

a) Minicomputer Model

Several minicomputers connected together.

b) Workstation Model

Users have personal workstations connected through a network.

c) Workstation-Server Model

Servers provide services to client workstations.

d) Processor Pool Model

Processors are dynamically allocated.

e) Hybrid Model

Combination of workstation and processor pool model.

---

2. Architectural Model

Defines interaction between components.

a) Client-Server Model

Clients request services from servers.

Advantages

- Centralized management

- Better security

Disadvantages

- Server bottleneck

---

b) Peer-to-Peer Model

All nodes act as both clients and servers.

Advantages

- Decentralized
- Scalable

Disadvantages

- Difficult security management

---

c) Multi-tier Architecture

Includes:

- Presentation layer
- Application layer
- Database layer

---

3. Fundamental Model

Describes core properties.

a) Interaction Model

Deals with communication delays and synchronization.

b) Failure Model

Studies various failures:

- Crash failure
- Omission failure
- Byzantine failure

c) Security Model

Deals with:

- Authentication
- Authorization
- Encryption

Conclusion

Distributed systems are designed to provide scalability, transparency, resource sharing, and fault tolerance. Different system models help in understanding hardware organization, software interaction, and system behavior.

\==========

Q3. Define distributed systems. Discuss their goals and challenges.

Definition

A distributed system is a collection of independent computers that appears to users as a single coherent system.

(Continue with goals from previous answer and challenges/issues from Q1.)

\==========

Q4. Explain the Client-Server Model in distributed systems.

Client-Server Model

The Client-Server model is a distributed architecture in which clients request services and servers provide services.

Components

1. Client

- Sends request
- Waits for response
- User-facing application

Examples

- Web browser
- Mobile app

---

2. Server

- Processes requests
- Manages resources
- Provides services

Examples

- Web server
- Database server

Working of Client-Server Model

1. Client sends request.
2. Server receives request.
3. Server processes request.
4. Server sends response.
5. Client displays result.

Types of Client-Server Architecture

1. Two-tier Architecture

Client communicates directly with database server.

2. Three-tier Architecture

- Client layer
- Application layer
- Database layer

3. Multi-tier Architecture

Used in enterprise applications.

Advantages

- Centralized control
- Better maintenance
- Improved security
- Resource sharing

Disadvantages

- Server bottleneck
- Single point of failure
- High server load

Applications

- Banking systems
- Email systems
- Web applications
- Cloud computing

Conclusion

The client-server model is one of the most widely used distributed architectures because it supports centralized management, scalability, and efficient resource sharing.

\==========

Q5. What is Middleware? Discuss services offered by middleware.

Definition

Middleware is software that acts as an intermediary layer between operating systems and distributed applications.

It hides the complexity of distributed systems and provides common services.

Functions of Middleware

- Communication management
- Data management
- Security services
- Naming services
- Synchronization

Services Offered by Middleware

1. Communication Services

Provides message passing and remote communication.

Examples

- RPC
- RMI
- Message Queues

---

2. Naming Services

Maps names to resources.

Example

DNS

---

3. Security Services

Provides:

- Authentication
- Authorization
- Encryption

---

4. Transaction Services

Ensures reliable execution of distributed transactions.

5. Persistence Services

Stores and retrieves data reliably.

---

6. Concurrency Control

Coordinates simultaneous access to resources.

---

7. Directory Services

Maintains resource information.

---

8. Fault Tolerance Services

Handles failures and recovery.

Advantages of Middleware

- Platform independence
- Simplified application development
- Better interoperability
- Improved scalability

Examples of Middleware

- CORBA
- Java RMI
- Web Services
- Message-Oriented Middleware

Conclusion

Middleware simplifies distributed application development by providing common services such as communication, security, synchronization, and transaction management.

==========

MODULE 2 – COMMUNICATION

Q1. Differentiate between RPC and RMI.

Basis	RPC	RMI
Full Form	Remote Procedure Call	Remote Method Invocation
Language Support	Language independent	Java specific
Communication	Procedure calls	Object-oriented method calls
Parameter Passing	Primitive data types	Objects can be passed
Programming Style	Procedural	Object-oriented
Platform	Multiple platforms	Java platform
Complexity	Simpler	More complex
Stub Usage	Client and server stubs	Stub and skeleton
Object Support	No	Yes
Example	Sun RPC	Java RMI

Conclusion

RPC is suitable for procedural distributed applications, whereas RMI supports distributed object-oriented programming.

\==========

Q2. Define Remote Procedure Call (RPC). Explain the working of RPC with diagram.

Definition

Remote Procedure Call (RPC) is a communication mechanism that allows a program to execute a procedure on a remote system as if it were a local procedure call.

RPC hides communication details from the programmer.

Components of RPC

1. Client
2. Client Stub
3. RPC Runtime System
4. Server Stub
5. Server

Working of RPC

Step 1

Client calls local procedure.

Step 2

Client stub converts parameters into message format (marshalling).

Step 3

RPC runtime sends request through network.

Step 4

Server stub receives request and converts message into parameters (unmarshalling).

Step 5

Server procedure executes.

Step 6

Result returned to server stub.

Step 7

Server stub marshals result.

Step 8

Response sent back to client.

Step 9

Client stub unmarshals response.

Step 10

Result returned to client application.

Diagram

Client → Client Stub → RPC Runtime → Network → Server Stub → Server

Server → Server Stub → RPC Runtime → Network → Client Stub → Client

---

Features of RPC

- Transparency
- Simplicity
- Location independence
- Efficient communication

Advantages

- Easy programming model
- Distributed transparency
- Code modularity

Disadvantages

- Network dependency
- Communication overhead
- Failure handling complexity

Applications

- Distributed databases
- File systems

- Cloud computing
- Web services

Conclusion

RPC simplifies distributed programming by enabling remote service invocation like local procedure calls.

\==========

Q3. Explain message communication models in distributed systems.

Message Communication Model

Message communication involves exchange of messages between processes.

Types of Communication

1. Transient Synchronous Communication

- Sender waits until receiver accepts message.
- Message lost if receiver inactive.

Advantages

- Simple synchronization

Disadvantages

- Blocking communication

---

2. Transient Asynchronous Communication

- Sender continues after sending.
- Message discarded if receiver unavailable.

Advantages

- Faster communication

Disadvantages

- Possible message loss

---

3. Persistent Synchronous Communication

- Message stored until delivered.
- Sender waits for acknowledgment.

Advantages

- Reliable delivery

Disadvantages

- Higher storage overhead

---

4. Persistent Asynchronous Communication

- Message stored permanently.
- Sender proceeds immediately.

Advantages

- High reliability
- Non-blocking

Disadvantages

- Complex implementation

Comparison Table

Type	Sender Blocks	Message Stored	Reliability
Transient Synchronous	Yes	No	Medium
Transient Asynchronous	No	No	Low
Persistent Synchronous	Yes	Yes	High
Persistent Asynchronous	No	Yes	Very High

Conclusion

Different communication models are selected depending on synchronization requirements, reliability, and system performance.

\==========

Q4. Differentiate between Message-Oriented and Stream-Oriented Communication.

Basis	Message-Oriented	Stream-Oriented
Data Unit	Messages	Continuous stream
Structure	Discrete packets	Continuous flow
Synchronization	Less strict	Time-dependent
Reliability	Easier	Complex
Examples	Email, RPC	Audio/Video streaming
Latency Requirement	Moderate	Very low
Data Preservation	Message boundaries preserved	Boundaries not preserved
Communication Style	Asynchronous possible	Usually continuous

Conclusion

Message-oriented communication is suitable for discrete data transfer, whereas stream-oriented communication is suitable for multimedia applications.

\==========

Q5. Discuss the role of group communication in distributed systems.

Group Communication

Group communication allows a process to send messages to multiple processes simultaneously.

Types

1. One-to-Many (1\:M)

One sender communicates with multiple receivers.

Example

Video conferencing.

---

2. Many-to-One (M:1)

Multiple senders communicate with one receiver.

Example

Online voting systems.

Importance

- Efficient communication
- Fault tolerance
- Replication support
- Scalability
- Synchronization

Applications

- Distributed databases
- Multiplayer gaming
- Collaborative systems
- Cloud computing

Advantages

- Reduced communication overhead
- Better coordination
- Faster information dissemination

Disadvantages

- Complex synchronization
- Message ordering problems
- Reliability challenges

Conclusion

Group communication is essential for coordination, replication, and collaborative processing in distributed systems.

==========

MODULE 3 – SYNCHRONIZATION

Q1. Explain Logical Clock and Lamport’s Algorithm.

Logical Clock

A logical clock is a mechanism used to order events in a distributed system without relying on physical time.

Need for Logical Clocks

Distributed systems:

- Have no global clock.
- Experience network delays.
- Need event ordering.

Logical clocks establish causal relationships.

Lamport’s Logical Clock Algorithm

Rules

Rule 1

Each process maintains a counter.

Rule 2

Increment counter before every event.

Rule 3

When sending message, attach timestamp.

Rule 4

Receiver updates clock:

Clock = max(local clock, received timestamp) + 1

Example

Suppose:

- P1 timestamp = 5
- P2 timestamp = 8

P1 sends message with timestamp 5. P2 receives it.

Updated clock at P2: max(8,5)+1 = 9

Advantages

- Maintains event ordering
- Simple implementation
- Supports synchronization

Disadvantages

- Cannot determine actual physical time
- Concurrent events difficult to detect

Applications

- Distributed databases
- Mutual exclusion
- Event synchronization

Conclusion

Lamport’s logical clock algorithm provides a reliable way to order distributed events logically.

\==========

Q2. Explain Suzuki-Kasami Broadcast Algorithm.

Introduction

Suzuki-Kasami is a token-based distributed mutual exclusion algorithm.

Only the process holding the token can enter the critical section.

Data Structures

1. RN Array

Request numbers.

2. LN Array

Last executed request numbers.

3. Queue

Stores requesting processes.

Working

Step 1

Process requests critical section.

Step 2

Broadcasts REQUEST message.

Step 3

Token holder checks request.

Step 4

Token transferred.

Step 5

Process enters critical section.

Step 6

After execution, token passed to next requester.

Advantages

- No deadlock
- Fair scheduling
- Efficient under heavy load

Disadvantages

- Token loss problem
- Broadcast overhead

Message Complexity

- Worst case: N messages

Conclusion

Suzuki-Kasami algorithm efficiently achieves distributed mutual exclusion using token passing.

\==========

Q3. Explain Maekawa’s Algorithm and properties of Quorum Set.

Introduction

Maekawa’s algorithm reduces message complexity using quorum sets.

A process communicates only with a subset of processes.

Quorum Set Properties

1. Intersection Property

Any two quorum sets must intersect.

2. Inclusion Property

Each process belongs to at least one quorum.

3. Equal Size Property

All quorum sets should have equal size.

Working

Step 1

Process sends REQUEST to quorum members.

Step 2

If members free, REPLY sent.

Step 3

After receiving all replies, process enters CS.

Step 4

After execution, RELEASE message sent.

Advantages

- Reduced message complexity
- Better scalability

Disadvantages

- Deadlock possibility
- Complex implementation

Message Complexity

Approximately $3\sqrt{N}$ messages.

Conclusion

Maekawa’s algorithm improves efficiency using quorum-based permissions.

\==========

Q4. Explain Bully Election Algorithm.

Election Algorithm

Used to select coordinator after failure.

Bully Algorithm

Highest process ID becomes coordinator.

Working

Step 1

Process detects coordinator failure.

Step 2

Election message sent to higher-ID processes.

Step 3

If no response, sender becomes coordinator.

Step 4

Coordinator message broadcasted.

Example

Processes: P1, P2, P3, P4 If P4 crashes:

- P2 starts election.
- P3 responds.
- P3 becomes coordinator.

Advantages

- Simple algorithm
- Fast recovery

Disadvantages

- High message overhead
- Multiple elections possible

Conclusion

The bully algorithm selects the highest-priority active process as coordinator.

\==========

Q5. Ricart-Agrawala Algorithm for Mutual Exclusion.

Introduction

Ricart-Agrawala algorithm is a permission-based mutual exclusion algorithm.

Working

Step 1

Process sends timestamped REQUEST.

Step 2

Receiving process replies if:

- Not interested in CS OR
- Sender has higher priority.

Step 3

Process enters CS after receiving all replies.

Step 4

After execution, deferred replies sent.

Advantages

- Lower message overhead
- No coordinator required

Disadvantages

- Process failure affects system
- High waiting time

Message Complexity

2(N−1) messages.

Optimization

Uses timestamps to reduce unnecessary communication.

Conclusion

Ricart-Agrawala algorithm optimizes distributed mutual exclusion using timestamp-based permissions.

\==========

MODULE 4 – RESOURCE AND PROCESS MANAGEMENT

Q1. Compare Load Sharing, Task Assignment, and Load Balancing.

Basis	Load Sharing	Task Assignment	Load Balancing
Objective	Avoid idle processors	Assign tasks optimally	Equalize workload
Decision Time	Dynamic	Static/Dynamic	Dynamic
Migration	Possible	Limited	Frequent
Complexity	Medium	Low	High
Goal	Improve utilization	Efficient allocation	Uniform distribution
Flexibility	Moderate	Low	High

Load Sharing

Distributes workload among systems.

Features

- Prevents idle resources.
- Improves throughput.

Task Assignment

Assigns tasks permanently.

Features

- Low migration cost.
- Simpler implementation.

Load Balancing

Balances workloads dynamically.

Features

- Improves response time.
- Maximizes resource usage.

Conclusion

Each strategy improves performance differently depending on system requirements.

\==========

Q2. Explain Process Migration and its significance.

Process Migration

Process migration is transferring a running process from one machine to another.

Reasons

- Load balancing
- Fault tolerance
- Better resource utilization
- Reduced execution time

Types

1. Preemptive Migration

Running process moved.

2. Non-preemptive Migration

Process moved before execution.

Steps

1. Suspend process.

2. Transfer state.
3. Restore process.
4. Resume execution.

Advantages

- Better performance
- Reduced overload
- Increased reliability

Disadvantages

- Migration overhead
- Security issues
- Complex implementation

Conclusion

Process migration enhances distributed system efficiency and resource utilization.

\==========

Q3. Explain Code Migration and its techniques.

Code Migration

Movement of code between machines during execution.

Techniques

1. Weak Mobility

Only code transferred.

2. Strong Mobility

Code + execution state transferred.

Models

a) Remote Evaluation

Code sent to remote machine.

b) Code on Demand

Code downloaded when needed.

c) Mobile Agent

Program migrates autonomously.

Challenges

- Security
- State transfer
- Platform heterogeneity
- Resource access

Advantages

- Flexibility
- Reduced communication
- Better scalability

Conclusion

Code migration improves flexibility and performance in distributed systems.

\==========

MODULE 5 – CONSISTENCY, REPLICATION AND FAULT TOLERANCE

Q1. Explain Data-Centric Consistency Models.

Consistency Model

Defines rules for accessing replicated shared data.

Types of Data-Centric Consistency Models

1. Strict Consistency

Any read returns most recent write.

Advantage

Highest consistency.

Disadvantage

Impossible in practical distributed systems.

---

2. Sequential Consistency

Operations executed in same order on all nodes.

---

3. Causal Consistency

Causally related writes observed in same order.

---

4. FIFO Consistency

Writes from same process seen in order.

---

5. Weak Consistency

Consistency guaranteed only after synchronization.

---

6. Release Consistency

Uses acquire and release operations.

---

7. Entry Consistency

Synchronization associated with variables.

Comparison

Model	Consistency Level	Performance
Strict	Very High	Low
Sequential	High	Medium
Causal	Medium	Better
FIFO	Medium	Better
Weak	Low	High

Conclusion

Consistency models balance correctness and performance in distributed systems.

\==========

Q2. Explain Client-Centric Consistency Models.

Client-Centric Consistency

Focuses on consistency observed by individual clients.

Types

1. Monotonic Read Consistency

A client never reads older values after newer ones.

---

2. Monotonic Write Consistency

Writes by a client executed in order.

---

3. Read Your Writes Consistency

A client always sees its own updates.

---

4. Writes Follow Reads Consistency

Writes based on latest reads.

Advantages

- Better user experience
- Suitable for mobile systems
- Improved scalability

Disadvantages

- Weak global consistency
- Synchronization complexity

Conclusion

Client-centric consistency provides practical consistency for distributed applications.

\==========

Q3. Explain Fault Tolerance and Process Resilience.

Fault Tolerance

Ability of a system to continue operating despite failures.

Goals

- Reliability
- Availability
- Data integrity

Techniques

1. Replication

Duplicate resources maintained.

2. Checkpointing

Save process state periodically.

3. Recovery

Restore system after failure.

4. Redundancy

Additional components added.

Process Resilience

Ability of processes to survive failures.

Methods

- Process groups
- Failure detection
- Logging
- Restart mechanisms

Failure Models

1. Crash Failure

System stops functioning.

2. Omission Failure

Messages lost.

3. Timing Failure

Responses delayed.

4. Byzantine Failure

Arbitrary incorrect behavior.

Conclusion

Fault tolerance and process resilience ensure reliable distributed system operation.

\==========

MODULE 6 – DISTRIBUTED FILE SYSTEMS

Q1. Features of DFS and Model File Service Architecture.

Distributed File System (DFS)

A DFS allows users to access files stored on distributed machines as if they are local.

Features of DFS

1. Transparency

Location and access hidden.

2. Scalability

Supports large number of users.

3. Reliability

Uses replication and backups.

4. High Availability

Files accessible despite failures.

5. Security

Authentication and authorization.

6. Concurrency

Multiple users can access files.

7. Performance

Caching improves speed.

Model File Service Architecture

Components

1. Client Module

Handles user requests.

2. File Server

Stores and manages files.

3. Directory Server

Maintains file locations.

Working

1. Client requests file.
2. Directory server locates file.
3. File server sends file.
4. Client accesses file locally.

Advantages

- Resource sharing
- Centralized management
- Improved collaboration

Disadvantages

- Network dependency
- Security concerns

Conclusion

DFS provides transparent, scalable, and reliable file access in distributed environments.

\==========

Q2. Explain HDFS in detail.

Hadoop Distributed File System (HDFS)

HDFS is a distributed file system designed for large-scale data storage and processing.

Architecture

1. NameNode

- Master server
- Maintains metadata
- Tracks block locations

2. DataNode

- Stores data blocks
- Handles read/write operations

3. Secondary NameNode

- Creates checkpoints
- Assists NameNode

Features

- Fault tolerance
- Scalability
- High throughput
- Data replication
- Large file support

Working

1. File divided into blocks.
2. Blocks stored on DataNodes.
3. Replicas created.
4. NameNode manages metadata.

Advantages

- Handles huge datasets
- Reliable storage
- Cost-effective

Disadvantages

- Not suitable for small files
- Single NameNode bottleneck

Applications

- Big Data analytics
- Data warehousing
- Machine learning

Conclusion

HDFS is a reliable and scalable distributed storage system for big data applications.

\==========

Q3. Compare AFS and NFS.

Basis	AFS	NFS
Full Form	Andrew File System	Network File System
Scalability	High	Moderate
Caching	Whole file caching	Block caching
Performance	Better for large networks	Better for small networks
Security	Strong	Moderate
Server State	Stateful	Stateless
File Sharing	Large-scale	LAN environments

Advantages of AFS

- Better scalability
- Reduced server load
- High availability

Advantages of NFS

- Simpler implementation
- Easy configuration

Conclusion

AFS is suitable for large distributed environments, while NFS is better for smaller network-based file sharing.

\==========

Q4. Explain file-caching schemes used in distributed file systems.

File Caching

Caching stores frequently used data locally to improve performance.

Types of Caching

1. Server-side Caching

Cache maintained at server.

---

2. Client-side Caching

Cache maintained at client.

Caching Schemes

1. Write-Through Cache

Updates immediately written to server.

Advantage

High consistency.

Disadvantage

Slower performance.

---

2. Write-Back Cache

Updates written later.

Advantage

Better performance.

Disadvantage

Possible inconsistency.

---

3. Cache Validation

Checks whether cached data is valid.

Advantages of Caching

- Faster access
- Reduced network traffic
- Better scalability

Disadvantages

- Consistency issues
- Cache coherence problems

Conclusion

Caching significantly improves DFS performance while introducing consistency management challenges.

\==========

IMPORTANT EXAM TIPS

Frequently Asked Questions

Very Important Topics

1. RPC
2. Lamport Logical Clock
3. Suzuki-Kasami Algorithm
4. Maekawa Algorithm
5. Data-Centric Consistency Models
6. HDFS
7. Load Balancing vs Load Sharing vs Task Assignment
8. DFS Features and Architecture

Answer Writing Strategy for 10 Marks

1. Start with definition.
2. Explain concept in points.
3. Draw neat diagram wherever possible.
4. Explain working step-by-step.
5. Mention advantages and disadvantages.
6. Add applications/examples.
7. Write conclusion.

END OF NOTES